

第1章 数据类型与变量

建议学时:3-5 小时 | 难度:★★★☆☆ | 读者背景:已掌握 C 语言

🎯 本章学习目标

- 掌握 JavaScript 的 8 种内建类型,弄清"C 里的 int 到 JS 里变成什么"这一核心迁移
- 理解 **number** 只有双精度浮点一种数值类型,没有 int,并掌握 BigInt 的适用场景
- 彻底分清 **undefined** 与 **null**、**typeof** 的著名坑、字符串的 UTF-16 编码模型
- 熟练使用 let/const 声明变量,理解 var 的提升(TDZ)与 const 的真实含义
- 掌握显式类型转换(Number/String/Boolean)与隐式转换的基本方向
- 会用数组与对象组织数据、用对象冻结模拟枚举(对应 C 的 enum)
- 理解"C 中怎么做→JS 中怎么做"的对照:编译期类型 → 运行时动态类型

💡 从 C 到 JavaScript:本章主题如何迁移

在 C 语言中,每一个变量在声明时就确定了它的类型与大小: `int x` 是 4 字节有符号整数, `char s[100]` 是 100 字节字符数组,类型是编译期的铁律, `sizeof` 随手可得。这些铁律保证了性能,但也带来了隐患——类型转换错误、越界、未初始化内存,一切都要程序员自己负责。

JavaScript 走的是完全相反的路: **类型属于值,不属于变量**。同一个变量可以先放整数、再放字符串、再放对象,完全合法。这被称为**动态类型**。代价是性能与"编译期帮你抓错"的能力,换来的是极高的表达力与灵活性。

你不需要担心内存:JS 有垃圾回收,没有 `malloc/free`,没有悬垂指针,没有数组越界写坏内存。你需要担心的变成了另一类问题:类型判断失误、隐式转换带来的诡异结果(如 `0 == ''` 为真)、`undefined` 满天飞。

生产中的 JS(Node.js 20+ 与浏览器)几乎都运行在 V8 引擎上,引擎内部会做 **隐藏类(hidden class)**与**即时编译(JIT)**优化——你的变量在运行期被观察到"总是同一个类型"时,代码会跑得和静态类型语言一样快。所以"动态类型"不等于"随意类型":同一变量保持同一类型的写法,既正确又高效。

本章的目标是:把 C 的类型心智模型整体迁移到 JS。读完后,你应该能在心里快速回答"这个 C 类型在 JS 里对应什么、该怎么安全地写"。先立总纲:数字只有一个(加一个大整数),字符串是 UTF-16,布尔是真的,空值有两个,其余全是对象。

🧰 前置知识自查

- C 的基本类型:int/char/float/double 的字节数与取值范围、有符号/无符号
- 变量的声明、初始化、作用域(块作用域与函数作用域)
- 字符串:char 数组、'\0' 结尾、strlen 与字节数
- enum 枚举、typedef 类型别名
- sizeof 运算符与类型大小的直觉

本章学习路线

1. **第一阶段(类型总览)**:读 §1.1 与 §1.2,先在脑中建立"8 种类型"的完整地图,把 C 类型逐一对号入座
2. **第二阶段(声明与转换)**:读 §1.3–§1.5,亲手在 Node REPL 里运行每个转换示例,观察输出
3. **第三阶段(字符串与复合类型)**:读 §1.6–§1.8,练习模板字符串与常用 API
4. **第四阶段(实验)**:独立完成章末实验"学生信息登记与成绩统计",用全部类型
5. **验收标准**:不看资料,能写出 `typeof` 对 8 种类型各自的输出,并说出 `undefined` 与 `null` 的 3 个区别

本章知识导图

- 第1章 数据类型与变量
 - §1.1 类型总览:C 类型 → JS 类型对照表
 - §1.2 `number` 与 `BigInt`:双精度浮点、整数安全范围
 - §1.3 变量声明:`let/const/var`、提升与 TDZ
 - §1.4 字面量与常量:数字/字符串/布尔/模板字符串、`const` 语义
 - §1.5 类型转换:显式转换与隐式转换的坑
 - §1.6 字符串:UTF-16、`length` 陷阱、常用 API
 - §1.7 复合类型入门:数组、对象、枚举模拟
 - §1.8 `undefined` 与 `null` 专题 + 类型别名(无)

§1.1 类型总览:C 的类型在 JS 里变成什么

直觉先行:一张表完成迁移

C 有 30 多种基本类型组合(`int`、`unsigned long`、`float`、`char*`.....);JS 只有 8 种,其中对象是一种"引用类型",其余 7 种是"原始类型"(primitive)。下表是本章最重要的对照表,建议反复看。

C 的写法	JS 的对应	说明
int / short / long	number	全部合并为双精度浮点,没有 int!
unsigned long long	bigint	超过 $2^{53}-1$ 的整数用 BigInt(字面量加 n)
float / double	number	double 是唯一浮点,float 不存在
_Bool(或 stdbool.h 的 bool)	boolean	C 没有真正的 bool,JS 有 true/false
char(字符)	string(单字符字符串)	JS 没有 char 类型,字符是长度为 1 的字符串
char[] / char*	string	字符串不可变,自带长度,无 '\0' 结尾
struct / union	object	对象,属性可动态增删
数组 int a[10]	Array(数组对象)	动态长度,可混合存放任意类型
指针 int* p	引用(值本身)	对象天然按引用共享,无指针运算
NULL 指针	null / undefined	两个空值,语义不同(见 §1.8)
enum	对象 + Object.freeze	用常量对象模拟,见 §1.7
typedef	(无)	JS 没有类型别名,类型跟随值

八种内建类型一览:`number`(双精度浮点)、`bigint`(任意精度整数)、`string`(UTF-16 字符串)、`boolean`、`undefined`、`null`、`symbol`(唯一标识,常用于对象键)、`object`(含数组、函数、日期等一切引用类型)。函数是对象的一种,但 `typeof` 函数 会返回 `'function'` ——这是历史遗留的特例。

⚠ 常见误区 1.1:typeof null 返回 'object'

这是语言自诞生就存在的 bug,无法修复(修复会破坏海量存量代码)。`typeof null === 'object'` 为 true,但 `null` 是原始类型。判断空值请直接用 `x === null`,不要依赖 `typeof`。

1.1.1 typeof 运算符:每个类型输出什么

```
// 在 Node.js 20+ 或浏览器控制台运行
console.log(typeof 42);           // 'number'
console.log(typeof 3.14);         // 'number' (没有 float,全是 number)
console.log(typeof 42n);          // 'bigint'
console.log(typeof 'hello');      // 'string'
console.log(typeof true);         // 'boolean'
console.log(typeof undefined);    // 'undefined'
console.log(typeof null);         // 'object' ← 著名的 bug,记住它!
console.log(typeof Symbol('id')); // 'symbol'
console.log(typeof {});           // 'object'
console.log(typeof []);           // 'object' ← 数组也是 object
console.log(typeof function (){}); // 'function' ← 函数特例
```

⚠ 常见误区 1.2:typeof 判不了"未声明的变量"就报错?

不会报错。`typeof undeclaredVar` 即使变量完全不存在也安全返回 `'undefined'`,这是 `typeof` 唯一的安全特性,生产代码里常用于特征检测,例如判断 `typeof process !== 'undefined'` 来区分 Node 与浏览器环境。

§1.2 number 与 BigInt:没有 int 的世界

C 的 `int` 加 1 溢出会回绕(有符号溢出是未定义行为);JS 的 `number` 加 1 溢出会变成 `Infinity`。因为 `number` 是 IEEE 754 双精度浮点,它能精确表示的最大整数是 $2^{53} - 1 = 9007199254740991$,即 `Number.MAX_SAFE_INTEGER`。超过这个范围的整数运算会悄悄丢失精度——这是生产事故的高发区(如订单号、ID、时间戳纳秒)。

```
// 双精度带来的"整数不精确"现象
console.log(0.1 + 0.2);           // 0.30000000000000004 ← 浮点误差,和 C 一样
console.log(9007199254740992);    // 9007199254740992
console.log(9007199254740992 + 1); // 9007199254740992 ← 精度丢了,+1 无效!
console.log(Number.MAX_SAFE_INTEGER); // 9007199254740991
console.log(Number.isSafeInteger(9007199254740992)); // false

// 浮点比较的正确姿势:用极小差值(epsilon)
function nearlyEqual(a, b) {
  return Math.abs(a - b) < Number.EPSILON * Math.max(Math.abs(a), Math.abs(b));
}
console.log(nearlyEqual(0.1 + 0.2, 0.3)); // true

// 大整数:用 BigInt,字面量末尾加 n
const big = 9007199254740993n;
console.log(big + 1n);             // 9007199254740994n
console.log(typeof big);           // 'bigint'
```

⚠ 常见误区 1.3:BigInt 与 number 不能混算

`1n + 1` 直接抛 `TypeError: Cannot mix BigInt and other types`。必须显式转换: `Number(1n) + 1` 或 `1n + BigInt(1)`。另外 `BigInt` 不能用于 `Math` 对象方法,也不能与浮点字面量混用。生产建议:需要大整数的字段(如雪花 ID)全程用 `BigInt`,或统一转字符串传输(JSON 不支持 `BigInt`)。

1.2.1 特殊数值:NaN、Infinity 与 -0

JS 有四个"奇特"的 `number` 值,生产中必须认识:**NaN**(Not a Number,唯一的非自等值:NaN 不等于任何数包括它自己)、**Infinity**、**-Infinity** 与 **-0**。除以 0 不会崩溃,而是返回 `Infinity`——这与 C 的"除零是未定义行为/崩溃"截然不同,需要 **显式检查**。

```
console.log(1 / 0);                // Infinity
console.log(0 / 0);                // NaN
console.log(NaN === NaN);          // false ← 经典坑
console.log(Number.isNaN(NaN));    // true ← 正确的判断方式
console.log(isNaN('abc'));         // true ← 全局 isNaN 先转字符串,有坑!
console.log(Number.isNaN('abc'));  // false ← 正确的 Number.isNaN
console.log(0 === -0);              // true(=== 分不出正负零)
console.log(Object.is(0, -0));     // false ← Object.is 能区分
console.log(Number.isFinite(Infinity)); // false
```

§1.3 变量声明:let / const / var

C 里变量声明只分"有初始化/无初始化";JS 里声明要回答两个问题:可变吗(const vs let)与作用域(块作用域 vs 函数作用域)。现代生产代码的黄金法则:默认 `const`,需要重新赋值才用 `let`,永远不用 `var` (历史遗留代码除外)。

对比项	C	JS 的 let/const
未初始化变量	局部变量是垃圾值(UB)	值是 undefined,不会读到垃圾
作用域	块作用域(C99 起)	块作用域(if/for/{} 内)
作用域外访问	编译错误	运行时 ReferenceError
重复声明	同作用域内报错	let/const 重复声明报 SyntaxError
声明前使用	可以(值未定义)	TDZ,报 ReferenceError(见下)

1.3.1 声明前使用:TDZ(暂时性死区)

let/const 声明的变量在声明语句执行之前处于 **暂时性死区** (Temporal Dead Zone, TDZ), 任何读取都会抛 ReferenceError。这比 C 的"未初始化垃圾值"严格得多,本质是语言替你抓错。

```
// console.log(x); // ❌ 报错:ReferenceError: Cannot access 'x' before initialization
let x = 10;
console.log(x);    // 10

// var 则是"提升"到函数顶部,值为 undefined,不报错——这正是 var 的坑
console.log(y);    // undefined(不报错,因为 var 提升了声明)
var y = 20;
```

★ 重点结论:const 的"不变"是什么

const obj = { n: 1 } 里, const 锁死的是绑定(不能重新赋值),不是内容。obj.n = 2 合法, obj = {} 报错。类比:C 里 int *const p 锁死指针本身,不锁死指向的对象。需要连内容都锁死时用 Object.freeze (浅冻结)。

```
const URL_BASE = 'https://api.example.com';
// URL_BASE = 'x'; // ❌ TypeError: Assignment to constant variable

const config = { retries: 3 };
config.retries = 5;    // ✅ 合法:修改属性不违反 const
console.log(config.retries); // 5
// config = {};    // ❌ 重新赋值报错

const list = [1, 2, 3];
list.push(4);    // ✅ 合法:数组内容可改
console.log(list);    // [ 1, 2, 3, 4 ]
```

§1.4 字面量与常量

1.4.1 数字字面量

C 的进制写法(0x、0、0b 等)与后缀(L、U、f)在 JS 中大部分保留,但 **没有后缀字母**,大整数用 n 后缀。数字分隔符 `_` 是 ES2021 新特性,用于提高大数字可读性。

```
const hex = 0xFF;           // 255,十六进制
const oct = 0o17;           // 15,八进制(0o 前缀,注意不是 C 的 0 前缀)
const bin = 0b1010;         // 10,二进制
const sci = 1.5e3;           // 1500,科学计数法
const sep = 1_000_000;       // 1000000,数字分隔符,纯可读性
const big = 12345678901234567890n; // BigInt 字面量
// 注意:没有 float/double 之分,3.14 与 3 都是 number
console.log(hex, oct, bin, sci, sep, typeof big);
```

1.4.2 字符串字面量与模板字符串

C 只有双引号一种字符串;JS 有 **三种**:单引号、双引号(二者等价,生产规范通常统一其一)、**模板字符串**(反引号,支持换行与 `${表达式}` 插值,对应 C 的 `sprintf` 家族)。

```
const s1 = '单引号';        // 单引号
const s2 = "双引号";        // 双引号,等价
const s3 = `多行
字符串`;                    // 模板字符串:可换行

const name = '张伟';
const score = 92;
// 模板插值 ≈ C 的 sprintf("%s 得分 %d", name, score)
const report = `${name} 得分 ${score} 分`;
console.log(report);         // 张伟 得分 92 分

// 转义序列与 C 类似
const path = 'C:\\Users\\me'; // 双反斜杠
const line = '第一行\n第二行';
console.log(line);
```

工程建议 1.1:拼接字符串用模板字符串,不要用 +

大量 `'a' + b + 'c'` 拼接可读性差且容易踩隐式转换的坑。一律使用模板字符串;需要性能的批量拼接(如循环内拼 1 万次以上)改用数组 `push + join` 或 `Array.from` ——V8 对字符串拼接有优化,但模板字符串仍是最佳默认。

§1.5 类型转换:C 是"你说了算",JS 是"我说了算"

C 的转换是 **显式为主**: `(int)x` 一句话,编译器按规则截断;隐式转换(整型提升、`int→double`)规则清晰。JS 的 `number/string/boolean` 之间的隐式转换规则 **诡异且频繁触发** (运算符 `+`、`==` 等),所以生产铁律: **显式转换,不要依赖隐式**。隐式转换细节在第2章讲透,本章先掌握显式转换。

```
// 显式转换:Number() / String() / Boolean()
console.log(Number('42'));           // 42
console.log(Number(''));              // 0 ← 空串变 0,坑!
console.log(Number(' 12 '));          // 12,自动去首尾空格
console.log(Number('12abc'));         // NaN ← 不是 12!全串解析失败
console.log(Number(true));            // 1
console.log(Number(null));            // 0
console.log(Number(undefined));       // NaN

console.log(String(42));              // '42'
console.log(String(null));            // 'null'
console.log(String([1, 2, 3]));       // '1,2,3'

console.log(Boolean(0));              // false
console.log(Boolean(''));            // false
console.log(Boolean('0'));            // true ← '0' 是非空字符串!

// parseInt 家族:部分解析,与 Number 完全不同
console.log(parseInt('42abc'));       // 42,解析到非法字符为止
console.log(parseInt('0x1F'));        // 31,自动识别 0x
console.log(parseInt('10', 2));       // 2,第二个参数是基数(radix)
console.log(parseInt('08'));          // 8(老浏览器曾按八进制解析出 0)
console.log(parseFloat('3.14abc'));  // 3.14
```

⚠ 常见误区 1.4:parseInt 必须显式给基数

生产代码请永远写 `parseInt(s, 10)`。虽然现代引擎已不把 '08' 当八进制,但代码评审规范通常要求显式基数,防止历史行为差异。另外 `parseInt` 用于"用户输入→整数"时,记得先检查 `Number.isFinite` 结果,NaN 不等于任何值, `parseInt('abc') === NaN` 是 false,比不出结果。

§1.6 字符串:不可变与 UTF-16

C 的字符串是 `char[]` 加 `\0`, `strlen` 数是字节;JS 的字符串是 **不可变对象** (任何"修改"都产生新字符串),没有 `\0`,自带 `length`。编码模型是 **UTF-16**:`length` 数的是 16 位码元(code unit),不是字符个数,也不是字节数。中文、英文、数字在 BMP 平面内都是一个码元,`length` 与字符数一致;但 **emoji、生僻字(增补平面)** 占用两个码元,`length` 会"多算"。


```

const zh = '你好世界';
console.log(zh.length);           // 4, 中文每字一个码元
console.log('hello'.length);      // 5, ASCII 一个码元
const emoji = '🍌';               // U+1F600, 增补平面
console.log(emoji.length);        // 2 ← 坑! 一个字符占两个码元
console.log([...emoji].length);    // 1 ← 展开运算符按码点拆分, 正确计数
console.log(emoji.codePointAt(0).toString(16)); // 1f600, 码点

// 常见 API 速查(与 C 对照)
console.log('hello world'.indexOf('world')); // 6, 查找位置(类似 strstr)
console.log('hello'.slice(1, 3));             // 'el', 子串(C 没有原生对应)
console.log('a,b,c'.split(','));             // ['a','b','c'], 拆分
console.log(['a', 'b'].join('-'));           // 'a-b', 拼接
console.log(' x '.trim());                   // 'x', 去首尾空白
console.log('abc'.toUpperCase());           // 'ABC'
console.log('abc'.replace('a', 'x'));        // 'xbc', 只替换第一处
console.log('3'.padStart(3, '0'));          // '003', 补零
console.log('hello'.includes('ell'));        // true
console.log('file.txt'.endsWith('.txt'));    // true
console.log('abc'.charAt(1));                // 'b', 等价 s[1]

```

⚠ 常见误区 1.5: 字符串不可变, 别学 C 改字符

`s[0] = 'x'` 在非严格模式下静默失败, 严格模式下抛错——总之改不了。要"改"就构造新串: `s.slice(0, i) + newChar + s.slice(i + 1)`。另外拼接百万次字符串时, 用数组收集再 `join`, 避免反复创建大对象。

§1.7 复合类型入门: 数组、对象与枚举模拟

1.7.1 数组: C 的 `int a[10]` → 动态数组

```

const nums = [10, 20, 30];          // 数组字面量
nums.push(40);                      // 尾部追加(类似 C 里手动 realloc)
nums.pop();                         // 移除尾部
console.log(nums.length);           // 3, 长度是动态的
console.log(nums[1]);               // 20, 按下标访问
// 数组可以装任何类型(C 做不到)
const mixed = [1, 'two', { name: 'three' }, [4]];
console.log(mixed[1]);              // 'two'
console.log(Array.from('abc'));     // ['a','b','c'], 从类数组转数组

```


1.7.2 对象:struct 的动态版

```
// 对象字面量 ≈ 匿名 struct,但属性可随时增删
const student = {
  id: 1001,
  name: '李雷',
  scores: [88, 92, 76],
};
student.age = 18; // 运行时加属性(C 做不到)
delete student.age; // 删除属性
console.log(student.name); // '李雷',点号访问
console.log(student['id']); // 1001,方括号访问(键可以是变量)
const key = 'name';
console.log(student[key]); // '李雷',动态键名

// 遍历对象属性
for (const k of Object.keys(student)) {
  console.log(k, student[k]);
}
```

1.7.3 枚举模拟:C 的 enum → 冻结对象

C 的 enum 在 JS 中没有直接对应。生产惯例:用 `const` 对象 + `Object.freeze` 模拟,配合 `Object.freeze` 使其不可修改。ES2020+ 还有更正式的 **Symbol** 枚举写法,但冻结对象最常用。

```
// 模拟 C 的 enum Color { RED, GREEN, BLUE }
const Color = Object.freeze({
  RED: 'red',
  GREEN: 'green',
  BLUE: 'blue',
});
// Color.RED = 'x'; // 严格模式下抛 TypeError,对象被冻结

function describe(c) {
  switch (c) {
    case Color.RED: return '红色';
    case Color.GREEN: return '绿色';
    case Color.BLUE: return '蓝色';
    default: return '未知颜色';
  }
}
console.log(describe(Color.RED)); // 红色
```

§1.8 undefined 与 null 专题、类型别名

💡 直觉先行:两个空值怎么分

undefined 表示"系统给的默认空":变量声明未赋值、函数没 return、对象没有某属性,JS 自动给你 undefined。
null 表示"程序员主动说的空":C 里你写 `return NULL`,JS 里你写 `return null`,表达"这里故意没有值"。分工:C 的空全靠 NULL 一个,JS 拆成两个——undefined 是"语言层面的缺席",null 是"业务层面的空"。

对比项	undefined	null
产生来源	语言自动(未初始化、缺失属性)	程序员显式赋值
typeof 结果	'undefined'	'object'(bug)
布尔转换	false	false
相等性	== null 为 true;=== 不相等	== undefined 为 true;=== 不相等
JSON 序列化	属性被丢弃	序列化为 null
参数默认值触发	会触发默认值	不会触发(传 null 就是 null)

```

let a;           // 声明未赋值
console.log(a);  // undefined
function f() {}  // 没有 return
console.log(f()); // undefined
const obj = {};
console.log(obj.missing); // undefined ← 缺属性也是 undefined

// 判断"没有值"的惯用法:== null 同时匹配 undefined 和 null
function safe(x) {
  if (x == null) return '空';
  return '有值';
}
console.log(safe(undefined)); // 空
console.log(safe(null));      // 空
console.log(safe(0));         // 有值 ← 0 不是空
console.log(safe(''));       // 有值 ← 空串也不是空

```

类型别名:JS 没有 typedef,也不需要——类型属于值,一个函数接受任何类型。第14章会介绍 JSDoc 注释与 TypeScript 如何补上"给类型起名"的能力,生产大型项目通常引入 TypeScript 或 JSDoc。

工程建议 1.2:函数参数永远不要传 undefined 当"空"

API 边界统一用 null 表示"没有值": `updateUser(id, null)` 语义清晰,JSON 序列化也会保留 null;undefined 在 JSON 中会被静默丢弃,可能导致"字段神秘消失"的线上事故。数据库字段的"空"也建议显式 null。

本章速查表

主题	速查要点
类型总数	8 种:number、bigint、string、boolean、undefined、null、symbol、object
int 去哪了	没有 int!整数也是 number(双精度),安全整数范围 $\pm 2^{53}-1$
大整数	BigInt,字面量 10n,不能与 number 混算
布尔	true/false;C 没有真 bool,JS 有
字符	没有 char 类型,字符是长度为 1 的 string
typeof null	'object'(语言 bug),判空用 <code>x === null</code> 或 <code>x == null</code>
NaN	<code>NaN !== NaN</code> ;判断用 <code>Number.isNaN</code>
除零	1/0 是 Infinity,不崩溃;0/0 是 NaN
声明	默认 const,要改值用 let,不用 var
TDZ	let/const 声明前访问抛 ReferenceError
const 语义	锁绑定不锁内容;要锁内容用 <code>Object.freeze</code>
字符串编码	UTF-16;length 是码元数,emoji 占 2
字符串不可变	所有"修改"产生新串,用 slice/split/join 组合
数字转整数	<code>parseInt(s, 10)</code> 显式基数;Number(s) 全串解析
undefined vs null	undefined 语言默认, null 业务空;JSON 只保留 null
枚举	<code>Object.freeze({...})</code> 模拟
类型别名	无;大型项目用 TypeScript/JSDoc

最小必会清单

- 能说出 8 种内建类型,并写出 typeof 对每种类型的输出(含 null 的 bug)
- 能解释为什么 `0.1 + 0.2 !== 0.3`,以及 `Number.isSafeInteger` 的用途
- 能说出 const 锁绑定不锁内容,并演示 `Object.freeze`
- 能解释 TDZ,并说明 var 提升为什么危险
- 能写出模板字符串插值,替代 C 的 `sprintf`
- 能说出 `Number('12abc')` 与 `parseInt('12abc')` 的区别
- 能解释字符串 length 对 emoji 的"多算"问题,并用 [...s] 修正
- 能写出 undefined 与 null 的 3 个区别(来源/typeof/JSON)
- 能用 `Object.freeze` 模拟枚举,并配合 switch 使用
- 能写出数组 push/pop/length 与对象增删属性的代码

自测题

Q 1. 写出以下代码在 Node.js 20 中的输出:`console.log(typeof 3.14, typeof 'a', typeof NaN, typeof [])`

✅ 参考答案

输出: 'number' 'string' 'number' 'object'。3.14 是 number(无 float);'a' 是字符串(无 char);NaN 是 number;[] 是 object。

Q 2. `const a = 5;` 之后执行 `a = 6` 会发生什么?`const obj = {x:1};` 之后执行 `obj.x = 2` 呢?

✅ 参考答案

`a = 6` 抛 `TypeError`(常量不能重新赋值); `obj.x = 2` 合法,`const` 只锁绑定不锁内容。

Q 3. 说明 `undefined` 与 `null` 在 `typeof`、JSON 序列化、默认参数三个场景下的差异。

✅ 参考答案

`typeof:undefined` 返回 'undefined',`null` 返回 'object'(bug)。JSON.stringify({a: undefined, b: null}) 结果是 '{"b":null}',`undefined` 属性被丢弃。默认参数 `function f(x = 1){}` :`f(undefined)` 得 1,`f(null)` 得 null。

Q 4. 为什么 `9007199254740992 + 1 === 9007199254740992`?如何安全计算这类大整数?

✅ 参考答案

9007199254740992 已超过 $2^{53}-1$ 的精确整数范围,双精度浮点表示不下 +1 的变化,结果被舍入回原值。改用 `BigInt`: `9007199254740992n + 1n`。

Q 5. `'ab😊cd'` 的 `length` 是多少?如何正确数出可见字符个数?

✅ 参考答案

`length` 为 6('ab' 2 个码元 + 😊 2 个码元 + 'cd' 2 个码元)。正确计数: `[...'ab😊cd'].length` 得 4,或用 `Array.from`、`Intl.Segmenter` (按字素,更精确)。

Q 6. `Number('')`、`Number('12abc')`、`parseInt('12abc')`、`Boolean('0')` 分别是什么?

✅ 参考答案

0、NaN、12、true。空串转数字是 0;Number 全串解析失败给 NaN;parseInt 部分解析;'0' 是非空字符串,转布尔是 true。

章末实验题:学生信息登记与成绩统计

实验 1:学生信息登记与成绩统计

实验目标:用全章类型与转换知识,实现一个命令行学生信息登记与成绩统计程序。

实验要求:

- 任务 1(覆盖:对象、数组):定义 Student 对象(id/name/scores),用数组保存多名学生
- 任务 2(覆盖:枚举模拟、常量):用 Object.freeze 定义成绩等级枚举(A/B/C/D/F)
- 任务 3(覆盖:number、BigInt、浮点精度):统计平均分,处理 0.1+0.2 类浮点误差(用 toFixed 或 epsilon 比较);为学号使用大整数演示 BigInt
- 任务 4(覆盖:字符串 API、模板字符串):姓名按模板字符串格式化输出,支持按姓名模糊查找(用 includes)
- 任务 5(覆盖:显式转换):从字符串输入解析分数(parseInt/Number),非法输入给出提示
- 任务 6(覆盖:undefined/null、typeof、TDZ、const):演示 const 绑定不可改、未初始化变量为 undefined、用 == null 防御空值

实验环境:Node.js 20+,在项目目录执行 `node student.js`。

第一步 **建数据层:**先定义 Color 式冻结枚举 `Grade`,再构造 3 名学生对象放入数组

第二步 **现统计函数:**平均分用 reduce 求和后除以 length,用 `Math.round(x * 100) / 100` 处理浮点误差

第三步 **现查找与等级判定:**写 `findByName`(用 includes)、`gradeOf`(用 if/else 链,覆盖全等级)

第四步 **分项:**把成绩解析封装成 `safeParseScore`,返回 null 表示解析失败(练习 null 语义)

✅ 参考答案要点

核心思路:全部数据用 `const` 声明(对象内容可变),防御空值用 `== null`,输出统一模板字符串。代码约 100 行,覆盖本章所有知识点。

```
// student.js — Node.js 20+ 运行:node student.js
'use strict';

// 1. 枚举模拟(覆盖:Object.freeze、const 对象)
const Grade = Object.freeze({
  A: 'A',
  B: 'B',
  C: 'C',
  D: 'D',
  F: 'F',
});

// 2. 学生对象与数组(覆盖:对象、数组、BigInt 学号)
const students = [
  { id: 2025001n, name: '李雷', scores: [88, 92, 76] },
  { id: 2025002n, name: '韩梅梅', scores: [95, 89, 90] },
  { id: 2025003n, name: '王芳', scores: [60, 70, 65] },
];

// 3. 平均分(覆盖:number、浮点精度、模板字符串)
function average(scores) {
  if (scores == null || scores.length === 0) return 0;
  const sum = scores.reduce((acc, s) => acc + s, 0);
  return Math.round((sum / scores.length) * 100) / 100;
}

// 4. 等级判定(覆盖:if/else、枚举使用、Boolean 真值)
function gradeOf(avg) {
  if (avg >= 90) return Grade.A;
  if (avg >= 80) return Grade.B;
  if (avg >= 70) return Grade.C;
  if (avg >= 60) return Grade.D;
  return Grade.F;
}

// 5. 按姓名查找(覆盖:字符串 API includes、undefined)
function findByName(name) {
  return students.find((s) => s.name.includes(name));
}

// 6. 安全解析分数(覆盖:显式转换、null 语义)
function safeParseScore(text) {
  const n = Number(text);
  if (!Number.isFinite(n)) return null;
  if (n < 0 || n > 100) return null;
  return n;
}

// 7. 打印成绩单(覆盖:模板字符串、for...of、解构)
function printReport() {
  console.log('===== 成绩单 =====');
  for (const s of students) {
    const avg = average(s.scores);
    console.log(`学号 ${s.id} | ${s.name} | 平均 ${avg} 分 | 等级 ${gradeOf(avg)}`);
  }
}
```



```
}

// 8. 主流程(覆盖:const/let、TDZ 概念、== null 防御)
function main() {
  printReport();

  const hit = findByName('梅梅');
  if (hit == null) {
    console.log('未找到学生');
  } else {
    console.log(`找到:${hit.name},平均 ${average(hit.scores)} 分`);
  }

  const demo = safeParseScore('78.5');
  console.log(`解析 '78.5' -> ${demo},等级 ${gradeOf(demo)}`);
  const bad = safeParseScore('abc');
  console.log(`解析 'abc' -> ${bad}(null 表示非法输入)`);
}

main();
```

设计说明:avg 先用 `Math.round(x*100)/100` 把浮点误差收敛到两位小数;findByName 返回 undefined 时用 `== null` 同时拦截 undefined 与 null;学号用 `BigInt` 演示大整数;全程序除循环变量外全部 `const`,符合生产规范。

下一章预告:第2章 运算符与表达式 —— 掌握 `===` 与 `==` 的全部差异、短路运算符返回操作数的行为、位运算的 32 位限制,这些是阅读任何 JS 代码的第一道坎。